

Configurer PathPlanner

Enregistrez un YAGSL-built SwerveDrive avec PathPlanner's AutoBuilder.

[PathPlanner](#) est un outil distinct de YAGSL — il planifie et suit des trajectoires multipoints (avec prise en compte des obstacles, des marqueurs d'événements et un éditeur d'interface graphique intégré), contrairement aux déplacements point à point de [driveToPose](#). Cette page décrit comment y intégrer un module YAGSL [SwerveDrive](#). Elle suppose que vous disposez déjà d'un robot de conduite créé à partir du [Tutoriel](#).

Cette page traite de la configuration de YAGSL/YAMS. Pour la création de chemins/automatismes, l'installation de l'application PathPlanner GUI autonome et les concepts généraux de PathPlanner, veuillez consulter [la documentation de PathPlanner](#).

1. Installez PathplannerLib

Ouvrez le panneau Dépendances du fournisseur WPILib dans VS Code (au même endroit où vous avez installé YAGSL — voir [Installer YAGSL](#)) et installez **PathplannerLib** à partir du catalogue.

2. Exportez la configuration de votre robot depuis l'interface graphique de PathPlanner.

Dans [l'application PathPlanner GUI](#), renseignez la masse, le moment d'inertie, les décalages des modules, le rayon des roues et les spécifications des moteurs d'entraînement de votre robot dans l'onglet Robot Config, puis exportez. Cette opération crée un fichier `deploy/pathplanner/settings.json` qui sera lu par `RobotConfig.fromGUISettings()` lors de l'exécution — vous n'avez pas besoin de modifier ce fichier manuellement ni de dupliquer ses valeurs en Java.

Il s'agit d'une **seconde description**, distincte de celle de YAGSL `modules/physicalproperties.json`, des propriétés physiques de votre robot. Le rapport de transmission et le diamètre des roues, notamment, figurent dans les deux descriptions. Si vous modifiez l'une (changement de roue, changement de rapport de transmission), mettez également à jour l'autre, sinon les trajectoires de PathPlanner et l'odométrie de YAGSL divergeront quant à la distance réellement parcourue par le robot. C'est une cause fréquente de dérive, invisible dans le JSON — consultez [la section PathPlanner de la documentation sur la dérive des systèmes de propulsion à pivot](#).

3. Registre [Autobuilder](#) une seule fois, juste après avoir créé le disque dur

Faites-le une seule fois, dans le constructeur de votre sous-système de déplacement latéral, juste après `SwerveParser.createSwerveDrive(...)` le retour de la fonction. Après cet

appel à **PathPlannerAuto**, les commandes et les chemins définis par l'interface graphique fonctionneront sans problème — aucun câblage supplémentaire n'est nécessaire par véhicule.

Java

```
import com.pathplanner.lib.auto.AutoBuilder;
import com.pathplanner.lib.config.PIDConstants;
import com.pathplanner.lib.config.RobotConfig;
import com.pathplanner.lib.controllers.PPHolonomicDriveController;
import edu.wpi.first.wpilibj.DriverStation;
import java.io.IOException;
import org.json.simple.parser.ParseException;

public class SwerveDriveSubsystem extends SubsystemBase {
    private SwerveDrive drive;

    public SwerveDriveSubsystem() {
        var cfg = new SwerveDriveConfig()
            .withStartingPose(new Pose2d(3, 3, Rotation2d.kZero))
            .withSubsystem(this)
            .withTelemetry(TelemetryVerbosity.HIGH);

        drive = new SwerveParser(new File(Filesystem.getDeployDirectory(),
"swerve/base"))
            .createSwerveDrive(cfg);

        try {
            setupPathPlanner();
        } catch (IOException | ParseException e) {
            throw new RuntimeException(
                "PathPlanner setup failed -- check deploy/pathplanner/settings.json
exists", e);
        }
    }

    private void setupPathPlanner() throws IOException, ParseException {
        AutoBuilder.configure(
            drive::getPose,                // robot pose supplier
            drive::resetOdometry,          // called if an auto defines a starting
pose
            drive::getRobotRelativeSpeed,  // ChassisSpeeds supplier -- MUST be
robot-relative
            (speedsRobotRelative, moduleFeedForwards) ->
                drive.setRobotRelativeChassisSpeeds(speedsRobotRelative),
            new PPHolonomicDriveController(
                new PIDConstants(5.0, 0.0, 0.0), // translation PID
        
```

```

        new PIDConstants(5.0, 0.0, 0.0) // rotation PID
    ),
    RobotConfig.fromGUISettings(), // reads
    deploy/pathplanner/settings.json
    () -> {
        // Field origin is always the blue alliance wall -- flip paths when on
        red.

        var alliance = DriverStation.getAlliance();
        return alliance.filter(a -> a ==
DriverStation.Alliance.Red).isPresent();
    },
    this // subsystem requirement for the
    generated commands
    );
}
}

```

Les `PPHolonomicDriveController` PID ci-dessus sont **indépendants** de `driveToPose` `SwerveDriveConfig.withTranslationController()/withRotationController()` — PathPlanner utilise son propre contrôleur et n'interagit jamais avec celui de YAGSL. Réglez-les séparément ; il n'est pas nécessaire qu'elles aient des gains identiques `.withRotationController()AutoBuilder`

4. Exécution de l'Auto

```

Java
public Command getAutonomousCommand() {
    return new PathPlannerAuto("New Auto");
}

```

Créez "New Auto", l'interface graphique de PathPlanner, déployez-la et liez-la à `getAutonomousCommand()` de la même manière que vous le feriez pour n'importe quelle autre commande autonome (`RobotContainer` fournisseur de commandes autonomes de PathPlanner, ou autre `Robot.autonomousInit()`). Les marqueurs d'événements placés dans l'interface graphique déclenchent des événements `EventTriggers` dans le code :

```

Java
new EventTrigger("EventMarker").whileTrue(Commands.print("Something"));

```

Dépannage

Si un PathPlanner parcourt une distance incorrecte, prend des virages au lieu d'aller tout droit ou se retrouve au mauvais endroit, consultez la [liste de contrôle PathPlanner dans Swerve Drive Drift](#) avant de procéder à un nouveau réglage ; la plupart de ces problèmes sont liés à une inadéquation de configuration à l'étape 2 ou à des gains PID qui n'ont jamais été réglés au-delà de leurs valeurs par défaut.